Projet MLSF — Partie I

Un solveur SAT pour le Meowdoku

Pedro DE BEM et Amanda FONTENELE

25 octobre 2026

Introduction

Ce projet consiste à construire un solveur SAT basé sur DPLL et à l'utiliser pour le Meowdoku. Nous présentons l'encodage des grilles en CNF, les principales décisions de programmation, les vérifications effectuées et la comparaison des heuristiques. Les fichiers Python et les résultats complets accompagnent ce rapport.

Organisation des fichiers

Le tableau suivant indique le rôle des principaux fichiers du rendu. Les commandes de reproduction des tests se trouvent dans `README.md`.

Fichier	Fonction
`meowdoku_to_dimacs.py`	Lecture d'une grille, construction des clauses et écriture du DIMACS (Q1).
`generate_grid.py`	Génération de grilles avec une solution connue (Q1).
`dimacs_to_pylist.py`	Lecture de DIMACS sous forme de listes Python (Q2).
`sat.py`	DPLL, DPLL avec valuation, heuristiques et interface de commande (Q2, Q4, Q6).
`heuristics.py`	Choix de branchement H_1, H_3, H_5 et H_7 dans leurs versions statiques et dynamiques (Q4).
`solve_meowdoku.py`	Conversion, résolution et contrôle d'une grille Meowdoku (Q3).
`benchmark.py`	Exécution des mesures avec limite de temps et vérification des réponses (Q5–Q6).
`Q1_script.py` à `Q6_script.py`	Scripts de test par question. Leurs sorties sont enregistrées dans `results/`.
`make_figures.py`	Génération du graphique comparant les heuristiques.
`image_grid.txt`	Grille d'exemple, au format de l'énoncé.
`meowdoku-instances/`	18 instances générées pour les essais Meowdoku (9 SAT et 9 UNSAT).
`public-instances/`	41 instances fournies par les enseignants.
`results/`	Résultats bruts des scripts et fichiers de solution.

Table 1 – Principaux fichiers et répertoires du rendu.

1 Question 1 — Modélisation et génération des grilles

1.1 Encodage en CNF

Pour une grille de N lignes, M colonnes et K régions $R_1, \ldots, R_K$, la variable $x_{i,j}$ vaut vrai si un chat occupe la case (i, j). Dans le fichier DIMACS, elle porte le numéro

$$\mathrm{id}(i, j) = (i - 1)M + j, \qquad 1 \leq i \leq N,\ 1 \leq j \leq M.$$

Nous obtenons les quatre familles de clauses suivantes, avec x_u pour la variable de la case u :

$$F_{\text{lignes}} = \bigwedge_{\substack{1\le i\le N\\ 1\le j<k\le M}} (\neg x_{i,j} \vee \neg x_{i,k}),$$

$$F_{\text{colonnes}} = \bigwedge_{\substack{1\le j\le M\\ 1\le i<k\le N}} (\neg x_{i,j} \vee \neg x_{k,j}),$$

$$F_{\text{régions}} = \bigwedge_{r=1}^{K} \left[\left(\bigvee_{u\in R_r} x_u \right) \wedge \left(\bigwedge_{\substack{\{u,v\}\subseteq R_r\\ u\neq v}} (\neg x_u \vee \neg x_v) \right) \right],$$

$$F_{\text{diag}} = \bigwedge_{\substack{1\le i<N\\ 1\le j<M}} \left[(\neg x_{i,j} \vee \neg x_{i+1,j+1}) \wedge (\neg x_{i,j+1} \vee \neg x_{i+1,j}) \right].$$

Les trois premières familles imposent au plus un chat par ligne et colonne, ainsi qu'exactement un chat par région. La dernière interdit les voisins en diagonale. Les voisinages horizontaux et verticaux sont déjà couverts par les contraintes de ligne et de colonne. Une valuation satisfait $F = F_{\text{lignes}} \wedge F_{\text{colonnes}} \wedge F_{\text{régions}} \wedge F_{\text{diag}}$ si et seulement si les cases associées aux variables vraies forment un placement légal. Comme il n'y a pas de variables auxiliaires, les placements légaux correspondent exactement aux modèles de F.

L'encodage comporte NM variables. En notant $s_r = |R_r|$, le nombre total de clauses (doublons compris) est :

$$C = \underbrace{N\binom{M}{2}}_{\text{lignes}} + \underbrace{M\binom{N}{2}}_{\text{colonnes}} + \underbrace{\sum_{r=1}^{K}\left(1+\binom{s_r}{2}\right)}_{\text{régions}} + \underbrace{2(N-1)(M-1)}_{\text{diagonales}}.$$

Le nombre de clauses est en $O(NM^2 + MN^2 + \sum_r s_r^2)$. Pour la grille 6×6 ci-dessous, on compte 36 variables et $90+90+128+50 = 358$ clauses. À titre de comparaison, une grille générée 50×50 comporte 196 650 clauses. Les éventuels doublons ne sont pas supprimés, car ils ne changent pas la satisfiabilité. Nous avons aussi testé les $6! = 720$ placements possibles à un chat par ligne et colonne sur la grille d'exemple : un seul respecte les contraintes. L'encodeur accepte $N \neq M$, même si le générateur se limite aux grilles carrées.

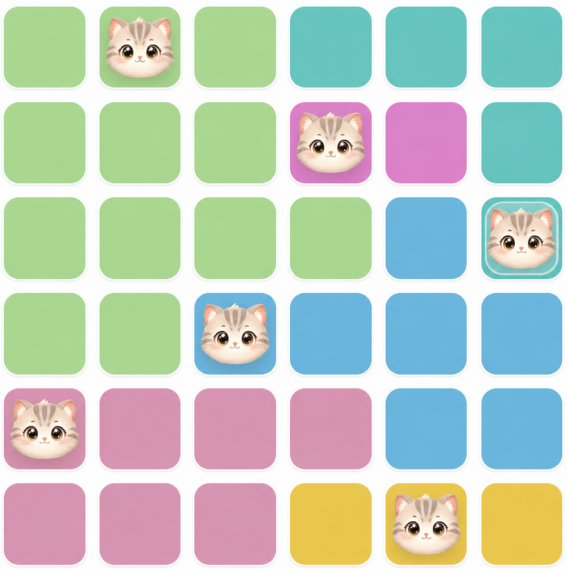


Figure 1 – Grille 6×6 utilisée comme exemple ; les chats montrent l'unique solution.

Le programme `meowdoku_to_dimacs.py` lit les dimensions, les noms des régions et la grille, contrôle

leur cohérence, puis produit le fichier `.cnf`. Le numéro des variables et le nombre de clauses sont écrits dans l'en-tête DIMACS.

1.2 Génération de grilles satisfiables

Le générateur `generate_grid.py` se limite au cas autorisé $N = M = K = n$. Il place d'abord un chat sur chaque ligne et chaque colonne, par retour arrière aléatoire, en imposant $|c_i - c_{i+1}| \geq 2$ aux colonnes de deux lignes consécutives. Chaque chat sert ensuite de point de départ à une région. Un parcours en largeur répartit les autres cases entre ces régions. Chaque région obtenue est connexe et contient le chat qui a servi à l'initialiser. Une solution est donc connue dès la génération de la grille. Une graine permet de reproduire les tirages. La méthode garantit l'existence d'une solution, mais pas son unicité : une grille 5×5 générée avec la graine 1 admet par exemple sept solutions. Les tailles 2 et 3 sont rejetées, car elles ne permettent pas de satisfaire toutes les contraintes.

2 Question 2 — Lecture DIMACS et DPLL

2.1 Représentation et algorithmes

Dans `dimacs_to_pylist.py`, une formule CNF est une liste de clauses, chaque clause étant une liste d'entiers signés. Une formule vide, `[]`, est satisfiable. Une formule qui contient une clause vide est insatisfiable. Le lecteur accepte les clauses réparties sur plusieurs lignes et contrôle notamment l'en-tête, les indices des variables, le marqueur final `0` et le nombre de clauses.

Les trois versions de DPLL suivent la même recherche. Elles appliquent d'abord la propagation unitaire, puis la règle du littéral pur. S'il reste des variables libres, le solveur en choisit une et essaie une valeur. En cas d'échec, il revient en arrière pour essayer la valeur opposée.

Dans `sat.py`, `dpll_status` renvoie seulement le statut SAT ou UNSAT, tandis que `dpll_model` construit aussi une valuation satisfaisante. Ces deux fonctions utilisent un choix aléatoire pour le branchement. La fonction `dpll_h` conserve les mêmes étapes, mais utilise une heuristique H pour choisir le littéral à essayer en premier. Elle enregistre également des compteurs pour les mesures de performance. Les trois fonctions partagent les opérations auxiliaires de simplification, de détection des clauses unitaires et de recherche de littéraux purs. Voici le pseudocode de `dpll_h`.

Procédure 1 : DPLL$_H(F)$ *statut et valuation*

Si F contient une clause vide, retourner $(\text{UNSAT}, \varnothing)$.
Si F ne contient plus de clause, retourner $(\text{SAT}, \varnothing)$.
Si F contient une clause unitaire $\{l\}$, ou à défaut un littéral pur l :
    $(s, v) \leftarrow \text{DPLL}_H(\text{Simplifier}(F, l))$.
    Si $s = \text{SAT}$, retourner $(\text{SAT}, v \cup \{l\})$, sinon $(\text{UNSAT}, \varnothing)$.
$l \leftarrow H(F)$ (littéral à essayer en premier).
Pour $l' \in (l, \neg l)$:
    $(s, v) \leftarrow \text{DPLL}_H(\text{Simplifier}(F, l'))$.
    si $s = \text{SAT}$, retourner $(\text{SAT}, v \cup \{l'\})$.
Retourner $(\text{UNSAT}, \varnothing)$.

Le littéral choisi indique aussi la valeur du premier essai. Si $H(F) = p$, nous essayons $p = 1$. Si $H(F) = \neg p$, nous commençons par $p = 0$. L'autre valeur n'est essayée qu'en cas d'échec.

`Simplifier(F,l)` retire les clauses qui contiennent l et supprime $\neg l$ des autres. La fonction crée une nouvelle liste de clauses, ce qui facilite le retour arrière, mais impose une copie à chaque affectation. Les valuations sont stockées dans des dictionnaires variable $\mapsto$ booléen. Pour la sortie DIMACS, les variables non affectées reçoivent la valeur *faux*. Si la formule ne contient ni clause unitaire ni littéral pur, le DPLL de base choisit aléatoirement son prochain littéral. Enfin, `run_deep` augmente la limite de récursion de Python et utilise une pile de 256 Mio pour les instances profondes. Sans cet ajustement,

`public-06`, qui comporte 1 363 variables, provoquait une erreur `RecursionError`.

Si V est le nombre de variables et L le nombre initial d'occurrences de littéraux, une borne simple pour le pire cas est $O(2^V L)$, puisque DPLL peut explorer un arbre exponentiel et parcourir la formule à chaque nœud.

2.2 Validation

Nous avons testé les 41 instances publiques trois fois avec chacune des deux versions de base, soit 246 exécutions limitées à 60 s. Les tests ont été lancés avec huit processus en parallèle et les durées indiquées sont des médianes. Les statuts retournés correspondent aux résultats attendus et les modèles des exécutions terminées satisfont les clauses. Les principales difficultés observées sont résumées ci-dessous.

Instance	Variables	Clauses	Statut	`dpll_model`	`dpll_status`
`public-03`	63	168	UNSAT	0/3, timeout	0/3, timeout
`public-06`	1363	3034	SAT	0/3, timeout	1/3, 0,5 s
`public-08`	100	900	UNSAT	3/3, 1,8 s	3/3, 0,7 s
`public-11`	729	21844	SAT	3/3, 36,8 s	3/3, 13,5 s
`public-12`	522	11636	SAT	1/3, 7,1 s	1/3, 3,3 s
`public-15`	100	850	UNSAT	3/3, 1,9 s	3/3, 2,6 s
`public-18`	459	7054	SAT	3/3, 21,0 s	3/3, 10,7 s
`public-19`	729	22060	SAT	3/3, 2,1 s	3/3, 3,3 s
`public-22`	100	850	UNSAT	3/3, 3,0 s	3/3, 3,0 s
`public-27`	181	3151	SAT	0/3, timeout	0/3, timeout
`public-30`	100	850	UNSAT	3/3, 3,5 s	3/3, 1,5 s

Table 2 – Instances lentes ou non systématiquement résolues. Le temps est la médiane des exécutions terminées.

La variabilité vient notamment du choix aléatoire du littéral. Par exemple, `public-12` a été résolue en 3,3 s dans un essai de `dpll_status`, mais a dépassé 60 s dans les deux autres. Nous ne pouvons donc pas conclure qu'une des deux versions est systématiquement plus rapide. Les temps restent indicatifs, car plusieurs processus ont tourné en parallèle.

3 Question 3 — Résolution complète du Meowdoku

Le programme `solve_meowdoku.py` relie les étapes précédentes. Pour rester conforme à l'interface demandée, il fait passer la formule par un fichier DIMACS temporaire, puis reconstruit la grille à partir du modèle.

Procédure 2 : Résolution d'une grille *entrée : fichier grille*

1 Lire la grille et construire sa formule CNF avec l'encodeur de la question 1.
2 Écrire la formule en DIMACS temporaire, puis la relire avec le lecteur de la question 2.
3 Résoudre avec DPLL. Si le résultat est UNSAT, signaler l'absence de solution.
4 Pour chaque variable vraie t, retrouver (i, j) avec $i = \lfloor (t-1)/M \rfloor + 1$ et $j = ((t-1) \bmod M) + 1$.
5 Vérifier les positions obtenues avec `check_solution`.
6 Écrire la grille d'entrée, les coordonnées par région et la grille marquée par `*`.

La fonction `check_solution` vérifie le résultat sur les cases, indépendamment des clauses CNF : un chat par région, au plus un par ligne et colonne, et aucune paire adjacente. Nous l'avons testée avec des placements illégaux (même ligne, même colonne et voisins diagonaux). Elle permet donc aussi de détecter une erreur d'encodage ou de conversion du modèle.

La grille de l'énoncé donne la solution attendue. Une grille 2×2 impossible est signalée comme insatisfiable. Des grilles générées (graines 1, 2, 3, de $n = 4$ à $n = 20$) ont aussi été résolues et contrôlées. Le tableau donne quelques temps représentatifs, avec une seule exécution par grille.

Taille n	Variables	Graine 1	Graine 2	Graine 3
4	16	0,07 s	0,07 s	0,07 s
8	64	0,09 s	0,08 s	0,08 s
12	144	0,14 s	0,10 s	0,08 s
15	225	0,16 s	1,21 s	5,07 s
20	400	2,58 s	0,41 s	7,00 s

Table 3 – Temps de résolution de grilles générées avec un branchement aléatoire.

4 Question 4 — Heuristiques de branchement

Nous avons retenu quatre heuristiques de l'énoncé, dont trois dans leurs versions statique et dynamique, soit sept configurations, auxquelles s'ajoute le choix aléatoire H_0.

Notons $\mathrm{occ}(l, F)$ le nombre d'occurrences du littéral l dans F, $\mathrm{occ}_k(l, F)$ le nombre d'occurrences dans des clauses de longueur k, et $k_{\min}$ la longueur minimale d'une clause de F.

Heuristique	Règle de sélection	Configurations
H_1 (PVL)	Première variable libre, affectée à vrai	H_1
H_3 (LP)	Littéral maximisant $\mathrm{occ}(l, F)$	H_{3s}, H_{3d}
H_5 (LFCM)	Littéral maximisant $\mathrm{occ}_{k_{\min}}(l, F)$	H_{5s}, H_{5d}
H_7 (LJW)	Littéral maximisant $\sum_k 2^{-k} \mathrm{occ}_k(l, F)$	H_{7s}, H_{7d}

Table 4 – Heuristiques implémentées dans `heuristics.py`.

La version *statique* calcule et classe les scores une fois sur la formule initiale. Elle retient ensuite le premier littéral du classement dont la variable apparaît encore dans la formule courante. La version *dynamique* recalcule les scores sur la formule simplifiée à chaque décision. En cas d'égalité, nous privilégions le plus petit indice de variable, puis le littéral positif.

Nous avons retenu H_3 comme règle de fréquence simple, H_5 pour privilégier les clauses les plus courtes, propices à la propagation unitaire, et H_7 pour pondérer toutes les tailles. H_1 sert de référence peu coûteuse. Si L désigne le nombre d'occurrences de littéraux dans la formule courante, un choix dynamique demande $O(L)$ pour recalculer les scores. Le classement statique est préparé une fois en $O(L_0 \log L_0)$, mais notre implémentation parcourt encore la formule à chaque décision pour identifier les variables restantes. Les huit configurations ont réussi les 88 tests de validation sur les 11 petites instances des enseignants (statuts et modèles vérifiés). Les instances plus grandes sont étudiées ensuite.

5 Question 5 — Comparaison des performances

5.1 Instances et protocole

Nous avons utilisé 30 instances publiques (19 SAT, 11 UNSAT) et 18 formules issues de Meowdoku. Les neuf formules Meowdoku SAT proviennent de grilles de tailles 10, 15 et 20, avec les graines 1 à 3. Pour créer neuf instances UNSAT, nous avons utilisé les grilles de tailles 8, 9 et 10 et ajouté une région X : il faudrait alors $n + 1$ chats pour seulement n lignes. Cette contradiction est indépendante du solveur.

Procédure 3 : Construction d'une instance Meowdoku UNSAT *taille n, graine s*

1 Générer une grille $n \times n$ satisfiable de graine s et conserver ses chats.
2 Prendre les trois premières cases sans chat et les affecter à une nouvelle région X.
3 Retourner la nouvelle grille, qui comporte $n + 1$ régions pour n lignes.

Nous prenons trois cases et non une seule pour ne pas rendre la contradiction immédiate par une simple propagation unitaire.

Chaque couple (instance, heuristique) a été exécuté une fois dans un processus séparé, avec une limite de 60 s, soit 384 essais. Jusqu'à dix processus ont tourné en parallèle. Nous avons compté les décisions et les retours arrière, et mesuré le temps CPU. Sur les instances communes, les retours arrière représentent 97,9 à 99,5 % des décisions. Nous ne les détaillons donc pas dans les tableaux. Les statuts et les modèles obtenus ont été vérifiés sans réponse erronée observée.

5.2 Résultats

	Publiques SAT	Publiques UNSAT	Meowdoku SAT	Meowdoku UNSAT	Total
H_0	17	10	9	9	45
H_1	18	10	8	9	45
H_{3s}	18	10	7	9	44
H_{3d}	18	10	9	9	46
H_{5s}	18	10	8	9	45
H_{5d}	19	10	9	9	47
H_{7s}	17	10	7	9	43
H_{7d}	17	10	9	9	45
Nombre d'instances	19	11	9	9	48

Table 5 – Instances résolues en moins de 60 s.

Le cas `public-03` n'a été résolu par aucune configuration. H_{5d} atteint le meilleur total, avec 47 instances sur 48. Les autres échecs concernent notamment `public-06`, `public-11`, `public-12` et certaines grilles 20×20, selon l'heuristique. Pour comparer l'effort sur les mêmes problèmes, le tableau suivant considère les 42 instances résolues par toutes les configurations : 16 publiques SAT, 10 publiques UNSAT, 7 Meowdoku SAT et 9 Meowdoku UNSAT.

	Pub. SAT		Pub. UNSAT		Meow. SAT		Meow. UNSAT		Part
	Déc.	CPU	Déc.	CPU	Déc.	CPU	Déc.	CPU	heur.
H_0	13,1k	70,0	9,4k	15,3	3,1k	2,1	24,1k	15,3	1,8 %
H_1	6,9k	66,4	5,2k	7,2	39,1k	29,0	1,4k	1,5	2,0 %
H_{3s}	24,1k	31,9	3,4k	5,0	33,2k	19,4	33,0k	15,5	3,3 %
H_{3d}	28,0k	50,9	2,3k	3,4	2,4k	3,4	48,4k	17,2	8,7 %
H_{5s}	6,8k	37,0	1,2k	0,5	33,2k	17,6	33,0k	18,5	2,9 %
H_{5d}	57,9k	77,3	1,0k	0,5	2,4k	3,5	48,4k	18,8	10,8 %
H_{7s}	5,3k	29,0	1,2k	0,6	33,2k	16,4	33,0k	15,2	3,0 %
H_{7d}	18,4k	62,7	1,0k	0,5	2,4k	3,8	48,4k	18,4	9,7 %

Table 6 – Décisions (milliers), temps CPU (secondes) et part mesurée dans l'heuristique, pour les 42 instances résolues par toutes les configurations.

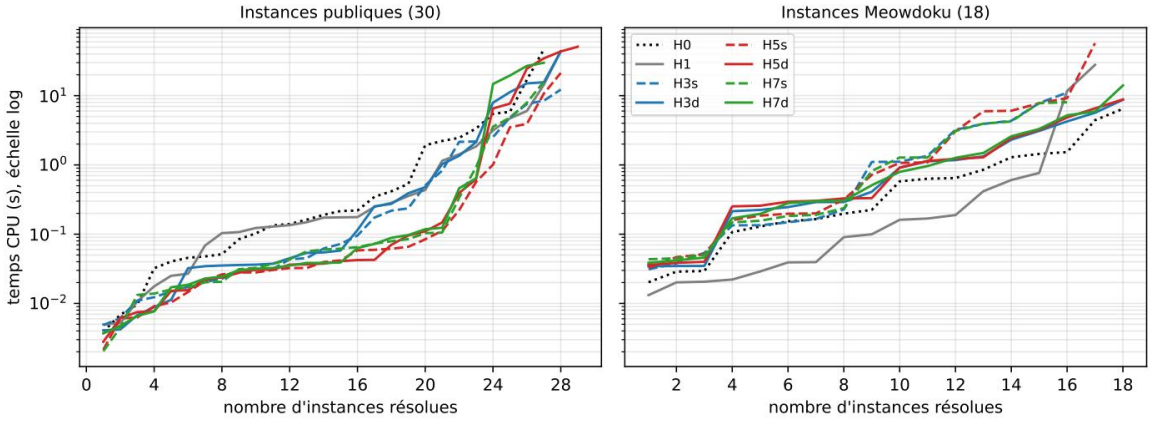


Figure 2 – Profils cumulés : pour chaque heuristique, temps CPU des instances résolues, triés par ordre croissant (échelle logarithmique). Plus une courbe est basse et longue, mieux c'est.

5.3 Analyse et limites

Les résultats dépendent nettement du type de formule. Sur les instances publiques UNSAT, H_5 et H_7 effectuent environ 1 000 à 1 200 décisions, contre 9 400 pour H_0, et prennent 0,5 à 0,6 s de CPU contre 15,3 s, soit un temps divisé par environ 25 à 30. Sur les publiques SAT, H_{7s} est le plus rapide de l'ensemble commun (29,0 s, contre 70,0 s pour H_0). En revanche, sur les sept grilles Meowdoku SAT communes, le choix aléatoire H_0 reste le plus rapide, avec 2,1 s. Une heuristique plus élaborée n'est donc pas toujours avantageuse.

Sur Meowdoku SAT, les variantes dynamiques de H_3, H_5 et H_7 passent à environ 2 400 décisions, contre 33 200 pour leurs variantes statiques. Elles sont 4 à 6 fois plus rapides sur les sept grilles communes et résolvent toutes les neuf grilles, contrairement aux variantes statiques. Le recalcul dynamique ne donne toutefois pas de gain sur les instances publiques SAT : pour H_5, le total atteint 77,3 s contre 37,0 s en statique. Sur Meowdoku UNSAT, H_1 arrive nettement devant les autres (1,5 s sur les neuf cas communs).

Les trois heuristiques dynamiques ont exactement les mêmes nombres de décisions sur les grilles Meowdoku. Une explication possible est la forte proportion de clauses binaires négatives, qui conduirait leurs scores à désigner les mêmes littéraux. Nous n'avons toutefois pas vérifié cette hypothèse. De même, nous supposons que l'ordre imposé par H_1 fait apparaître assez tôt la contradiction dans les grilles UNSAT, sans pouvoir le démontrer à partir de ces tests.

Le calcul de l'heuristique représente environ 9 à 11 % du temps CPU pour les variantes dynamiques, contre environ 3 % pour les statiques. Il peut être rentable sur certaines familles, surtout les grilles Meowdoku SAT, mais pas systématiquement. H_{5d} reste le meilleur choix selon le nombre d'instances terminées : 47 sur 48 contre 45 pour H_0.

Ces résultats doivent être lus avec prudence. Chaque couple (instance, heuristique) a été exécuté une seule fois, les choix de H_0 sont aléatoires et jusqu'à dix processus partageaient la machine. Les totaux du tableau ne portent que sur les instances résolues par les huit configurations et excluent donc les cas les plus difficiles. Enfin, nous n'avons pas mesuré séparément le nombre de propagations unitaires ni celui des éliminations de littéraux purs.

6 Question 6 — Interface et choix du mode optimisé

Le point d'entrée est `sat.py`. Il accepte les commandes suivantes :

```
python sat.py instance.cnf -mode dpll
python sat.py instance.cnf -mode optimized
```

```
python sat.py instance.cnf -output-model
```

Le mode par défaut est `dpll`. La sortie standard ne contient que `s SATISFIABLE` ou `s UNSATISFIABLE`. Avec `-output-model`, une instance SAT produit aussi une ligne `v` terminée par `0`, avec un littéral signé par variable (les variables non affectées sont mises à faux). Aucun modèle n'est écrit en cas d'UNSAT, et les erreurs sont envoyées sur `stderr`. Le mode `optimized` applique H_{5d}, choisi pour son nombre d'instances résolues, premier critère du classement. La simplification et les structures de données ne changent pas. Seul l'ordre de branchement diffère.

Mode	Réponses correctes (sur 41)	Temps cumulé des réponses
`dpll`	39	51,8 s
`optimized`	40	134,7 s

Table 7 – Essai des deux modes en ligne de commande (une exécution par instance, limite de 60 s).

Le mode `optimized` résout `public-27` en 40,7 s, alors que le mode de base dépasse 60 s. `public-03` reste non résolue. Sur les 39 instances communes, le mode optimisé est plus lent (94,0 s contre 51,8 s), notamment sur `public-06` (35,1 s contre 0,6 s) et `public-12` (25,9 s contre 12,4 s). Le mode `dpll` a pu bénéficier de tirages favorables, alors que `optimized` est déterministe. Nous avons privilégié le nombre d'instances résolues. Pour gagner en vitesse, il faudrait surtout remplacer les copies de clauses par une propagation incrémentale avec, par exemple, des *watched literals* et des affectations réversibles.

7 Bilan

L'encodage est vérifié de façon exhaustive sur la grille exemple et, indépendamment, par `check_solution` ; le solveur est correct sur toutes les instances disponibles (aucune réponse fausse sur les 41 instances publiques et les 18 formules Meowdoku). Ses faiblesses sont la vitesse (une instance publique non résolue, plusieurs résolues près de la limite) et le protocole de mesure (une exécution par couple, critères partiels), signalées ci-dessus. Le point le plus utile pour la suite serait de remplacer la propagation par copie par une propagation incrémentale.

Utilisation de l'intelligence artificielle

Nous avons utilisé ChatGPT pour comprendre certains concepts du cours, notamment l'algorithme DPLL et les heuristiques de branchement, ainsi que pour écrire et déboguer du code Python. Nous avons ensuite relu les propositions et vérifié le fonctionnement des programmes avec les tests du projet.

Environnement et reproduction

Machine	Acer Nitro ANV16-42-R3JW, Ryzen 7 260, 8 cœurs physiques (16 logiques), 14 Gio de RAM.
Système	Ubuntu 26.04.1 LTS, Python 3.14.4.
Parallélisme	Q2 : 8 processus, Q5 : 10, Q6 : 8 et Q3 : exécution séquentielle.

La machine diffère de celle de la compétition (Python 3.12, 4 CPU et 8 Gio dans Docker). La syntaxe du code a été contrôlée pour Python 3.12, sans exécution sur cette version. Les temps ne sont donc pas directement comparables. Les scripts `Q1_script.py` à `Q6_script.py` enregistrent les résultats dans `results/`. Les commandes pour relancer les tests se trouvent dans `README.md`.